

16. DQN

Overview

Deep Q-Networks approximate Q-values with neural nets, using experience replay and target networks for stability. We derive the TD loss, discuss overestimation bias, and map hyperparameters to CartPole training on CPU/GPU.

Lab: Lab 16.

Prior modules: Module 15.

Contents

1	Scaling Q-Learning	2
2	DQN Architecture	2
3	DQN Loss	2
4	Experience Replay	2
5	ϵ-Greedy and Double DQN	2
6	Limitations	2
7	Lab	3
8	Prioritized Replay	3
8.1	Rainbow Combine	3
9	Extended Intuition	3
10	Numerical Walkthrough	4
11	PyTorch Implementation Notes	4
12	Local GPU Constraints	4
13	Debugging Checklist	4
13.1	Replay Buffer Mechanics and DQN Variants	5
14	Practice Problems	5

Module track

This module builds on **Module 15**. Read those PDFs first. References to other modules appear as **Module NN** (not page numbers, because each module is its own PDF).

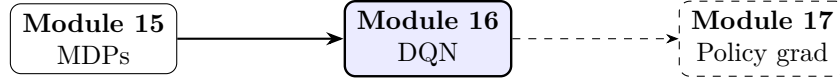


Figure 1: Module sequence (solid box: this PDF; dashed: typical next step).

1 Scaling Q-Learning

Tabular **Module 15** (equation) fails on high-dimensional states (images, Atari frames). DQN [3] approximates $Q(s, a; \theta)$ with a CNN.

2 DQN Architecture

Input stack of 4 grayscale frames $\rightarrow$ conv layers $\rightarrow$ fully connected $\rightarrow |\mathcal{A}|$ Q-values.

$$Q(s, a; \theta) \approx Q^*(s, a). \quad (1)$$

Two key innovations. Experience replay and target network stabilize bootstrapping from **Module 15** (Bellman equation).

3 DQN Loss

Sample minibatch (s, a, r, s') from replay buffer $\mathcal{D}$:

$$\mathcal{L}(\theta) = \mathbb{E}_{(s, a, r, s') \sim \mathcal{D}} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2 \right]. \quad (2)$$

θ^- is target network updated every C steps: $\theta^- \leftarrow \theta$.

4 Experience Replay

Store transitions in circular buffer capacity 10^6 ; uniform random sampling breaks temporal correlation. Improves sample efficiency over on-line updates from **Module 15** (Q-learning update).

5 ϵ -Greedy and Double DQN

Double DQN decouples selection and evaluation:

$$y = r + \gamma Q\left(s', \arg \max_{a'} Q(s', a'; \theta); \theta^-\right). \quad (3)$$

Reduces overestimation bias from max operator in (2).

6 Limitations

DQN handles discrete actions only; continuous control needs policy gradients (**Module 17** (policy gradient theorem)). Reward clipping and frame preprocessing are domain-specific engineering.

Table 1: DQN hyperparameters (Atari).

Hyperparameter	Value
Replay buffer size	10^6
Minibatch size	32
γ	0.99
Target update freq. C	10,000 steps
ϵ anneal	$1.0 \rightarrow 0.1$

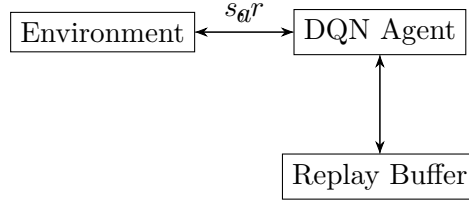


Figure 2: DQN with experience replay.

7 Lab

Lab 16 trains DQN on CartPole or Atari; compare convergence to tabular Q in **Lab 15**.

8 Prioritized Replay

Sample transition i with probability $P(i) \propto |\delta_i| + \epsilon$:

$$w_i = \left(\frac{1}{N} \cdot \frac{1}{P(i)} \right)^\beta, \quad \text{importance-weighted loss.} \quad (4)$$

Dueling architecture separates $V(s)$ and advantage $A(s, a)$: $Q(s, a) = V(s) + A(s, a) - \mathbb{E}_a A(s, a)$.

8.1 Rainbow Combine

Prioritized + Double + Dueling + Multi-step + Distributional + Noisy nets stack improvements over vanilla (2).

9 Extended Intuition

Deep Q-Networks replace the Q-table with $Q_\theta(s, a)$ parameterized by a neural network. Experience replay stores transitions (s, a, r, s') in a buffer and samples random mini-batches, breaking temporal correlation that destabilizes SGD. Target network $Q_{\theta-}$ uses frozen weights updated periodically so bootstrap targets do not chase a moving objective every step.

Loss for one sample:

$$\mathcal{L} = \left(r + \gamma \max_{a'} Q_{\theta-}(s', a') - Q_\theta(s, a) \right)^2. \quad (5)$$

Double DQN reduces overestimation by selecting a' with online net, evaluating with target net (optional extension).

Atari adds frame stacking, reward clipping, and convolutional encoders; CartPole uses a 2-layer MLP on 4-dim state. Exploration remains ϵ -greedy, often annealed over 100k to 1M frames.

Policy gradient methods in **Module 17** handle continuous action spaces where $\max_{a'}$ is impractical.

10 Numerical Walkthrough

CartPole-v1: state dim 4, 2 discrete actions, MLP $4 \rightarrow 128 \rightarrow 128 \rightarrow 2$.

Replay buffer capacity 100k transitions; batch $B = 64$, $\gamma = 0.99$, $\text{lr} = 10^{-3}$, target update every 1000 steps.

Single transition: s , action left, $r = 1.0$, s' not done. If $Q_{\theta-}(s', \cdot) = [10.2, 9.8]$, bootstrap $= 0.99 \cdot 10.2 = 10.098$. If $Q_{\theta}(s, \text{left}) = 8.5$, squared error $(1 + 10.098 - 8.5)^2 \approx 6.8$.

Training 500 episodes: moving average return crosses 200 (solved threshold) often by episode 150 to 300 with tuned ϵ schedule. Frame skip not used in CartPole; Atari would skip 4 frames and max-pool.

Memory: buffer $100k \times (4 + 1 + 1 + 4 + 1)$ floats negligible; network params $\approx 50k$.

11 PyTorch Implementation Notes

- Create env with `gymnasium.make("CartPole-v1")`.
- Step returns `obs, reward, terminated, truncated, info = env.step(a)`.
- Store `done = terminated or truncated` for bootstrap mask.
- Q-network outputs vector of Q-values per action; pick action ϵ -greedy on `q_values.argmax()`.
- Target computation:

```
with torch.no_grad():
    next_q = target_net(s_next).max(1)[0]
    target = r + gamma * next_q * (1 - done)
```

- Gather predicted Q: `q_sa = q_values.gather(1, a.unsqueeze(1)).squeeze(1)`.

```
loss = F.mse_loss(q_sa, target)
optimizer.zero_grad()
loss.backward()
optimizer.step()
if step % 1000 == 0:
    target_net.load_state_dict(q_net.state_dict())
```

12 Local GPU Constraints

CartPole DQN trains on CPU in seconds; GPU optional for habit. Atari CNN DQN on 1650: use frame size 84x84, replay on CPU RAM, batch 32; may need hours for 1M frames.

Mixed precision rarely needed for small MLP DQN; Atari benefits modestly from GPU conv forward.

13 Debugging Checklist

- Return stuck below 50: ϵ not decaying, or target net never updates.
- Q-values explode: lr too high; clip gradients or reduce lr to 10^{-4} .
- Loss zero immediately: replay buffer empty or same transition repeated; verify `buffer.sample`.
- Done flag ignored: bootstrap at terminal states inflates targets.
- Action always same: network outputs tied; check initialization and exploration.

13.1 Replay Buffer Mechanics and DQN Variants

The replay buffer is a FIFO queue: when capacity is exceeded, oldest transitions drop off, which slowly removes bias toward early poor policies. Uniform sampling assumes all transitions are equally informative; prioritized replay (optional extension) weights samples by TD error magnitude so surprising transitions get replayed more often. Store transitions as $(\mathbf{s}, \mathbf{a}, \mathbf{r}, \mathbf{s}', \text{done})$ tuples in float32; for Atari, stack 4 frames into $\mathbf{s}$ before pushing to keep Markov structure without storing full video.

Target network sync every C steps (1000 to 10000) trades stability for lag: too frequent sync chases a moving target; too rare sync slows learning. Double DQN decouples selection and evaluation:

$$y = r + \gamma Q_{\theta^-} \left(s', \arg \max_{a'} Q_{\theta}(s', a') \right). \quad (6)$$

This reduces systematic overestimation from max operator bias in vanilla DQN. Dueling architecture splits $Q(s, a) = V(s) + A(s, a) - \text{mean}_a A(s, a)$, helping states where action choice matters little (many Atari frames).

ϵ -greedy schedule: start $\epsilon = 1.0$, linear decay to 0.05 over first 50k to 100k steps for CartPole; hold small ϵ forever to avoid premature exploitation on stochastic envs. Frame stacking for Atari: each s_t concatenates grayscale frames $t - 3, \dots, t$ along channel dim; apply max over last two raw frames on s_t when a_t is fire button to reduce flicker. Reward clipping to $\{-1, 0, +1\}$ in Atari stabilizes Q magnitudes but removes fine-grained feedback; skip clipping on CartPole where rewards are already bounded. Gradient clipping at norm 10 before optimizer step prevents occasional TD target spikes from destabilizing weights when replay contains rare high-return trajectories. Compare final sample efficiency against tabular Q from **Module 15** on the same discretized CartPole grid to verify the network actually generalizes across similar states. n -step returns ($n = 3$) blend one-step TD and Monte Carlo: target uses reward sequence $r_{t+1}, \dots, r_{t+n}$ plus bootstrap $Q(s_{t+n})$; reduces bias at cost of slightly higher variance. Huber loss on TD error dampens impact of outlier targets compared to pure MSE; switch when Q-values occasionally spike above 100 on Atari runs. Soft target updates (Polyak averaging $\theta^- \leftarrow \tau \theta + (1 - \tau) \theta^-$) replace hard sync every C steps in some implementations; $\tau = 0.005$ is a common default. Set `torch.backends.cudnn.benchmark = True` for fixed-size Atari conv stacks; disable when input shapes vary per batch to avoid autotuner overhead. Minimum replay size before training starts: wait until buffer holds 1000+ transitions or Q overfits to correlated early episodes. Noisy nets (optional) add learned parametric noise to weights for exploration without ϵ -greedy; useful when action space is large and epsilon schedule is hard to tune. Log TD error histogram monthly in long Atari runs; bimodal distribution (many near zero, tail of spikes) suggests prioritized replay would help. Save Q-network weights every 50k frames; Atari runs crash from driver timeouts and replay buffer rebuild is expensive.

14 Practice Problems

Problem 1. Replay buffer size 50k, sample batch 64; expected unique transitions per batch (approx)?

Problem 2. Plot Q-value of best action at initial CartPole state over training steps.

Problem 3. Ablate target network (update every step); compare stability to periodic sync.

Problem 4. Compare tabular Q on discretized CartPole (**Module 15**) vs DQN sample efficiency.

Train in **Lab 16**; builds on **Module 15**; compare to policy gradients in **Module 17**.

References

- [1] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *ICLR*, 2015.

-
- [2] Volodymyr Mnih, Adria Puigdomenech Badia, Mehdi Mirza, et al. Asynchronous methods for deep reinforcement learning. In *ICML*, 2016.
 - [3] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, et al. Human-level control through deep reinforcement learning. *Nature*, 518:529–533, 2015.
 - [4] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018.
 - [5] Hado Van Hasselt, Arthur Guez, and David Silver. Deep reinforcement learning with double Q-learning. In *AAAI*, 2016.